

Arhitectura și Implementarea unui Sistem Distribuit de Planificare a Sarcinilor

Ghinoiu Cristina and Voștinar Oana

Academia Tehnică Militară "Ferdinand I"

Facultatea de Sisteme Informatice și Securitate Cibernetică

București, România

Acest articol prezintă proiectarea și implementarea unui sistem de planificare a sarcinilor (Task Scheduling System) bazat pe o arhitectură client-server. Sistemul permite utilizatorilor să programeze execuția unor comenzi la momente specifice, gestionând concurența și alocarea resurselor printr-un mecanism centralizat de planificare și execuție. Se discută aspecte cheie precum comunicarea între procese (IPC), sincronizarea firelor de execuție (multithreading), gestionarea proceselor prin primitive de sistem (fork, exec, signals) și algoritmi de planificare de tip Round-Robin cu preempție.

I. INTRODUCERE

Planificarea eficientă a proceselor reprezintă o componentă critică a sistemelor de operare, având impact direct asupra performanței și utilizării resurselor hardware. În contextul sistemelor distribuite, problema planificării capătă noi dimensiuni, necesitând mecanisme de coordonare a execuției la distanță și gestionarea concurenței între mulți utilizatori.

Această lucrare descrie proiectarea și implementarea unui sistem distribuit de planificare a sarcinilor (*Distributed Task Scheduling System*), care integrează primitive de sistem POSIX pentru controlul proceselor cu o arhitectură de rețea TCP/IP. Sistemul propus funcționează ca un planificator în spațiul utilizator (*user-space scheduler*), oferind funcționalități de execuție temporizată și management al concurenței, similare celor din nucleul sistemului de operare, dar extinse într-un mediu distribuit.

Contribuția principală a lucrării constă în implementarea unui mecanism hibrid de planificare care combină:

1. Un model de comunicare asincron bazat pe cozi de mesaje și protocol text proprietar.
2. Un algoritm de planificare Round-Robin cu suport pentru preempție, implementat prin manipularea semnalelor de sistem (SIGSTOP, SIGCONT).
3. Izolarea execuției prin grupuri de procese (**process groups**), asigurând stabilitatea serverului gazdă.

Obiectivele specifice ale implementării vizează decuplarea logică a componentelor (Client, Server, Executor), asigurarea consistenței datelor prin mecanisme de sincronizare *lock-free* și optimizarea utilizării procesorului în scenarii multi-utilizator.

II. LUCRĂRI CONEXE ȘI CONTEXT TEORETIC

Proiectarea sistemelor de planificare a evoluat de la utilitare locale la platforme distribuite complexe. Acest

proiect se situează la intersecția dintre programarea de sistem "low-level" și arhitecturile distribuite.

A. De la Cron la Sisteme Distribuite

În sistemele Unix, planificarea sarcinilor se bazează în mod tradițional pe daemonul cron, a cărui funcționalitate este limitată la execuții locale și la o granularitate temporală fixă. Soluțiile moderne de tip Job Scheduler distribuit, precum Jenkins sau Airflow, ascund în mare parte detaliile legate de sistemul de operare și se concentrează pe definirea și execuția fluxurilor de lucru (pipelines). Sistemul propus în această lucrare se poziționează între aceste soluții și mecanismele clasice de planificare, permițând execuția la distanță a sarcinilor, dar păstrând un control direct asupra proceselor și semnalelor POSIX. Această abordare oferă utilizatorului o vizibilitate mai bună asupra modului de execuție, comparativ cu soluțiile de nivel înalt.

B. Paradigme de Comunicare

În multe sisteme distribuite comerciale, precum gRPC sau Apache Thrift, comunicarea este realizată prin mecanisme de tip Remote Procedure Call, care ascund detaliile legate de rețea și simplifică interacțiunea dintre componente. Deși această abordare este convenabilă, ea limitează controlul asupra modului în care sunt gestionate conexiunile și datele transmise.

În cadrul acestui proiect, comunicarea dintre client și server este realizată folosind socket-i TCP furnizați de biblioteca SFML Network, peste care a fost definit un protocol propriu de schimb de mesaje. Această abordare permite menținerea unei conexiuni persistente și un control mai bun asupra fluxului de date dintre cele două componente. Clientul poate interoga serverul pentru a obține informații despre starea execuției sarcinilor și rezultatul acestora, cum ar fi codul de terminare al unui task. Comparativ cu soluțiile bazate pe HTTP, această metodă reduce complexitatea comunicației și este adecvată pentru un sistem de planificare a sarcinilor cu control direct

asupra execuției.

C. Mecanisme de Izolare

Platformele moderne de management al execuției aplicațiilor, precum Kubernetes și Nomad, se bazează pe tehnologii de containerizare (cgroups, namespaces) pentru izolarea sarcinilor. Proiectul de față explorează mecanismele fundamentale din spatele acestor abstracții, utilizând direct primitive POSIX pentru managementul grupurilor de procese (setpgid). Spre deosebire de un simplu Process Watcher (e.g. supervisor), care doar repornește procese, acest sistem implementează un algoritm activ de planificare a resurselor CPU („User-Space Time Slicing”) folosind semnalele SIGSTOP și SIGCONT pentru a suspenda și relua execuția, emulând comportamentul unui scheduler de sistem de operare într-un mediu distribuit.

III. ARHITECTURA ȘI DESIGNUL SISTEMULUI

Arhitectura sistemului propus adoptă o structură modulară, organizată pe niveluri de responsabilitate distincte, pentru a satisface atât exigențele de interactivitate ale utilizatorului final, cât și criteriile de performanță și stabilitate impuse backend-ului. Această organizare se reflectă direct în structura sistemului de fișiere și în procesul de construcție (build) automatizat.

A. Organizarea Proiectului

Arhitectura fizică a sistemului reflectă diviziunea logică a responsabilităților, facilitând mentenanța și dezvoltarea independentă a modulelor. Codul sursă este organizat în două directoare rădăcină distincte:

- **/Client:** Conține sursele interfeței grafice și logica de prezentare. Acest modul este complet decouplat de backend, singura legătură fiind protocolul de rețea.
- **/Server:** Găzduiește logica de business, împărțită la rândul ei în fișiere specializate pe funcțiuni: `server.cpp` (rețea), `scheduler.cpp` (planificare) și `executor.cpp` (procesare sistem).

Această separare permite utilizarea unor lanțuri de compilare (Makefiles) diferite, optimizate pentru nevoile fiecărei componente, așa cum este detaliat în Tabelul I.

TABLE I. Organizarea Proiectului

Componentă	Rol și Responsabilitate Arhitecturală
Client (Qt6)	Frontend: Captarea intenției utilizatorului.
Server (SFML)	Backend: Scheduler, Executor, persistență XML.
Build	Makefile: Dependențe (pthread, sfml, tinypng).

B. Analiza Procesului de Build

Procesul de compilare este unul hibrid, adaptat naturii diferite a celor două componente majore.

1. Lanțul de Compilare al Clientului (Meta-programming)

Pentru componenta Client, procesul de build implică etape complexe de generare de cod, specifice ecosistemului Qt:

- **Meta-Object Compiler (MOC):** Analizează fișierele de antet (ex: `MainWindow.h`) și generează cod C++ adițional (`moc_MainWindow.cpp`) pentru a permite mecanismele de *Runtime Introspection* și sistemul de Semnale/Sloturi. Fără acest pas intermediar, comunicarea asincronă din interfață ar fi imposibilă.
- **User Interface Compiler (UIC):** Transformă definițiile declarative XML ale interfeței (`MainWindow.ui`) în cod C++ imperativ (`ui_MainWindow.h`), separând designul vizual de logica funcțională.

2. Lanțul de Compilare al Serverului (System Linking)

Backend-ul necesită o strategie diferită, axată pe linkarea eficientă cu biblioteci de sistem de nivel jos:

- **POSIX Threading (-lpthread):** Esențial pentru suportul multithreading (Server/Scheduler/Executor).
- **Network Stack (-lsfml-network):** O abstractizare performantă peste socket-urile BSD.
- **Persistență (-ltinypng):** O bibliotecă minimalistă pentru parsing XML, aleasă pentru a minimiza amprenta de memorie a serverului.

Această separare strictă a fluxurilor de compilare (vizibilă în existența a două Makefile-uri distincte) garantează că modificările în logica de prezentare nu necesită recompilarea nucleului serverului, asigurând o mentenanță facilă.

IV. COMPONENTELE PRINCIPALE

A. Clientul (Interfața Grafică)

Aplicația client este dezvoltată utilizând framework-ul Qt, oferind o interfață grafică (GUI) intuitivă.

Tehnologii: C++, Qt Widgets, SFML Network (pentru comunicarea pe socket).

Funcționalități:

- **Autentificare:** Formulare dedicate pentru Login și Sign-up.
- **Dashboard:** Vizualizarea sarcinilor sub formă de liste (Active, Istoric).
- **Interacțiune:** Adăugarea, modificarea și ștergerea sarcinilor prin ferestre de dialog și meniuri contextuale.

Flux de date: Clientul capturează input-ul utilizatorului, îl serializează conform protocolului definit și așteaptă răspunsuri blocante sau asincrone de la server.

B. Serverul (Gestionarul de Conexiuni)

Serverul acționează ca un demon care ascultă pe portul 54000.

- **Model de Concurență:** Utilizarea modelului *Thread-per-Client*. Bucla principală (`server_main`) acceptă conexiuni noi folosind `sf::TcpListener`.
- **Multithreading:** Pentru fiecare conexiune acceptată, se creează un nou fir de execuție (`pthread_create`) care rulează funcția `session_loop`.
- **Responsabilități:**
 - Deserializarea mesajelor primite.
 - Validarea credențialelor prin intermediul componentei `AccountManager`.
 - Manipularea cererilor CRUD (Create, Read, Update, Delete) asupra sarcinilor, gestionând stocarea, modificarea și ștergerea acestora în format XML.
 - Notificarea Scheduler-ului la apariția unor sarcini noi prin setarea variabilei atomice `reload_needed`.

C. Planificatorul (Scheduler)

Componenta de planificare rulează într-un fir de execuție independent și asigură lansarea sarcinilor la momentul oportun.

- **Mecanism:** Polling periodic la un interval de 100ms.

Logică:

- Încarcă sarcinile din `waitingtasklist.xml` prin funcția `load_tasks_from_file`.
- Compară timestamp-ul curent (`time(0)`) cu cel planificat extras din structura `tm`.
- Dacă `timp_curent >= timp_planificat`, sarcina este extrasă și trimisă către Executor.
- Sarcinile mai vechi de o oră sunt considerate expirate și eliminate automat pentru a menține integritatea listei.

Listing 1. Logica de polling a Scheduler-ului

```
// scheduler.cpp
while (true) {
    if (reload_needed.load()) {
        reload_needed.store(false);
        load_tasks_from_file(tasks);
    }
    time_t now = time(0);
    // Verificare si trimitere catre coada
    // Executorului
    if (now >= task_time) {
        batch_tasks.push_back(execTask);
        queueCond.notify_all();
    }
    usleep(100000); // 100ms
}
```

D. Executorul (Engine-ul de Procesare)

Executorul gestionează resursele CPU și rularea efectivă a comenzilor prin procese de sistem.

- **Structură:** Un pool de 8 fire de execuție (*Workers*) care consumă sarcini dintr-o coadă partajată protejată de un mutex.
- **Algoritm de Planificare (Round-Robin pe Utilizatori):** Executorul menține o structură `std::map<user, deque<Task>`. Firele de execuție nu preiau pur și simplu următorul task, ci ciclează prin lista de utilizatori (`currentUserIndex`) pentru a asigura echitatea (Fairness). Astfel, un utilizator cu un volum mare de sarcini nu va bloca sistemul pentru ceilalți.

V. PROTOCOLUL DE COMUNICARE

Sistemul utilizează o arhitectură Client-Server robustă, bazată pe suita de protocoale TCP/IP pentru a asigura livrarea fiabilă a datelor. Comunicarea între componenta Client (interfața cu utilizatorul) și Server (gestionarul de sarcini) se realizează printr-un protocol proprietar, la nivelul aplicației, orientat pe mesaje text.

A. Structura Mesajelor

Protocolul este definit ca un schimb de șiruri de caractere (string-uri), unde câmpurile individuale de date sunt delimitate printr-un separator specific. Această abordare text-based facilitează atât dezvoltarea, cât și depanarea (debugging), permițând inspectarea directă a traficului de rețea.

Formatul general al unui mesaj este:

`COD_COMANDA||PARAMETRU_1||...||PARAMETRU_N`

Unde **COD_COMANDA** este un număr întreg care identifică tipul operațiunii solicitate, iar `||` este delimitatorul standard utilizat pentru separarea câmpurilor.

B. Tipuri de Mesaje și Comenzi

Protocolul definește un set strict de operațiuni, mapate pe coduri numerice, acoperind întregul ciclu de viață al unei sesiuni de utilizator.

1. Autentificare și Sesiune

- **Autentificare (Login - Cod 0):** Clientul trimite `0||username||parola`. Serverul verifică credențialele și, în caz de succes, răspunde cu codul 2 urmat de lista sarcinilor curente.
- **Înregistrare (Sign In - Cod 1):** Cererea are formatul `1||user||mail||pass`. Serverul validează unicitatea și returnează 0 pentru succes (declanșând și un email de confirmare).
- **Deconectare (Logout - Cod 2):** Mesajul 2 semnalează serverului încheierea voluntară a sesiunii, permițând eliberarea resurselor alocate firelor de execuție.

2. Managementul Sarcinilor

- **Adăugare Sarcină (Cod 3):** Incluziunea tuturor detaliilor necesare instanțierii obiectului `Task` pe server într-un singur mesaj: `3||Descriere||ZZ.LL.AAAA||HH:MM:SS||User||Prio`.
- **Interogare (Cod 4, 5, 6):** Protocolul permite filtrarea sarcinilor pentru optimizarea traficului: **Istoric (4)**, **Toate (5)** și **În Așteptare (6)**. Răspunsul serverului este o listă serializată.

- **Modificare și Ștergere (Cod 7, 8):** Operațiunile de actualizare utilizează ID-ul unic al sarcinii drept cheie de căutare. (Ex: `7||ID` pentru ștergere).

TABLE II. Tabela Codurilor de Operație (OpCodes)

OpCode	Descriere	Format Cerere (Client)
0/1	Login/SignUp	0 user pass / 1 user mail pass
3	Add Task	3 task_serialized
4	Tasks history	4 user
5	All Tasks	5 user
6	Waiting Tasks	6 user
7/8	Delete/Modify	7 id / 8 id task data...

VI. MANAGEMENTUL DATELOR ȘI SECURITATEA

Dincolo de execuția propriu-zisă a sarcinilor, infrastructura serverului trebuie să gestioneze persistența datelor și securitatea conturilor de utilizator.

A. Persistența Datelor (XML)

Pentru stocarea datelor structurate, s-a optat pentru formatul XML datorită structurii sale ierarhice și lizibilității umane (human-readable). Biblioteca `tinyxml2` este utilizată pentru operațiile de parsing și serializare.

- **waitingtasklist.xml:** Stochează sarcinile active, având attribute pentru identificare (ID), momentul execuției și prioritate.
- **accountslist.xml:** Funcționează ca o bază de date pentru utilizatori, stocând perechi de credențiale și adrese de email.
- **history.xml:** Menține un jurnal al execuțiilor finalizate, incluzând codul de exit returnat de procesul fiu, esențial pentru auditare.

B. Validarea și Securitatea Conturilor

Gestionarea conturilor este realizată prin componenta `AccountManager`. Validarea utilizatorilor la autentificare implică încărcarea fișierului XML și o căutare liniară pentru potrivirea perechii `user-parolă`. Deși simplistă, această abordare asigură consistența datelor prin reîncărcarea fișierului la fiecare cerere critică, evitând problemele de concurență (race conditions) la citire prin utilizarea unor instanțe locale de `XMLDocument` în fiecare fir de execuție.

C. Integrarea cu Sistemul de Operare pentru Notificări

Sistemul demonstrează capacitatea de a interacționa cu mediul gazdă prin componenta MailManager. La crearea unui cont nou, serverul nu implementează propriul client SMTP, ci apelează un script Python extern (send_mail.py) care gestionează comunicarea.

Aceasta se realizează printr-un apel `system()` care execută o comandă shell construită dinamic:

```
// MailManager.cpp
std::string command = "python3 send_mail.py \""
    + mail + "\" \"" +
        subject + "\" \"" + body +
        "\" \"" +
        SENDER_EMAIL + "\" \"" +
        SENDER_PASSWORD + "\" \""
    ;
system(command.c_str());
```

Script-ul Python utilizează biblioteca `smtpplib` pentru a trimite email-uri prin serverul SMTP al Gmail (`smtp.gmail.com:587`), cu autentificare TLS. Această metodă subliniază filozofia UNIX de "re-utilizare a uneltelor existente" și oferă flexibilitate sporită prin separarea logicii de trimitere a email-urilor într-un script independent, permițând extinderea funcționalității aplicației cu un efort minim de implementare.

VII. ANALIZA DETALIATĂ A IMPLEMENTĂRII ȘI SECVENȚE CRITICE DE COD

Această secțiune analizează soluțiile tehnice adoptate pentru problemele fundamentale ale sistemelor distribuite implementate: modelul de concurență hibrid, mecanismele de sincronizare fine-grained și algoritmul de planificare cu preemptare.

A. Modelul de Concurență: "Thread-per-Client" și "Worker Pool"

Serverul implementează un model de concurență hibrid. Pentru gestionarea conexiunilor de rețea, se utilizează o abordare *Thread-per-Client*, asigurând că operațiile de I/O lente ale unui client nu blochează restul sistemului.

Listing 2. server.cpp: Delegarea conexiunilor (Liniile 232-246)

```
for (;;)
{
    sf::TcpSocket* client = new sf::TcpSocket()
    ;
    auto st = listener.accept(*client);
```

```
if (st != sf::Socket::Done) {
    delete client;
    continue;
}

std::cout << "[server] sesiune noua\n";
pthread_t t;
// Crearea unui nou fir de executie pentru
    fiecare client
if (pthread_create(&t, nullptr, session_loop
    , client) != 0) {
    perror("pthread_create session");
    delete client;
} else {
    // Detasarea firului pentru eliberarea
        automata a resurselor la terminare
    pthread_detach(t);
}
}
```

Utilizarea `pthread_detach` este critică aici, eliminând necesitatea ca firul principal să apeleze explicit `pthread_join` și prevenind scurgerile de resurse (zombie threads) într-un server care rulează pe termen lung.

B. Sincronizarea și Evitarea Deadlock-urilor (RAII)

Într-un mediu multithreaded, accesul la resursele partajate (coada de sarcini, lista de utilizatori) trebuie protejat strict. Am implementat paradigma **Resource Acquisition Is Initialization (RAII)** printr-o clasă `LockGuard` personalizată, care garantează eliberarea mutex-ului chiar și în cazul apariției excepțiilor, prevenind astfel situațiile de *deadlock*.

Deși sistemul utilizează primitivele de nivel jos din biblioteca `pthread`, am ales să nu le expunem direct în logica de business. Am implementat un strat de abstracție C++ (Wrapper Classes) definit în `header.h`, care încapsulează gestionarea resurselor.

Listing 3. Wrapper C++ peste `pthread_mutex`

```
// header.h
class LockGuard {
private:
    Mutex& m_mutex;
public:
    LockGuard(Mutex& m) : m_mutex(m) {
        m_mutex.lock(); // pthread_mutex_lock
    }
    ~LockGuard() {
        m_mutex.unlock(); //
            pthread_mutex_unlock
    }
};
```

Această arhitectură face codul mai sigur (Exception Safety) și mai ușor de auditat.

Listing 4. executor.cpp: Sincronizare în bucla Worker-ului (Liniile 42-63)

```
{
    // Mutex-ul este blocat automat la crearea
    // obiectului LockGuard
    LockGuard lock(queueMutex);

    // Așteptare conditionată pentru eficiența
    // CPU (evita busy-waiting)
    queueCond.wait(queueMutex, []{
        for (const auto& user : userList) {
            if (!userQueues[user].empty())
                return true;
        }
        return false;
    });

    // Algoritmul Round-Robin pentru selecția
    // echitabilă a sarcinilor
    int n = userList.size();
    for (int i = 0; i < n; ++i) {
        int idx = (currentUserIndex + i) % n;
        const std::string& user = userList[idx];
        if (!userQueues[user].empty()) {
            t = userQueues[user].front();
            userQueues[user].pop_front();
            currentUserIndex = (idx + 1) % n; //
            // Rotirea indexului
            found = true;
            break;
        }
    }
} // Mutex-ul este deblocat automat aici (
// destructorul LockGuard)
```

C. Managementul Proceselor: Izolare și Preemptare

Nucleul sistemului îl reprezintă capacitatea Executorului de a rula comenzi arbitrare în mod controlat. Acest lucru se realizează prin primitivele de sistem POSIX fork, execvp, și setpgid.

1. Izolarea prin Grupuri de Procese (PGID)

Pentru a putea controla nu doar procesul lansat, ci și eventualii săi copii, am utilizat setpgid(0, 0). Aceasta plasează procesul într-un nou grup, izolat de server.

Listing 5. executor.cpp: Crearea și izolarea procesului (Liniile 69-78)

```
pid_t pid = fork();
if (pid == 0) {
    // Procesul Fiul (Child)
    setpgid(0, 0); // Crearea unui nou Process
    // Group ID

    const char* shell = "/bin/sh";
    std::vector<char*> args = { /* ... */ };

    // nlocuirea imaginii procesului cu
    // comanda utilizatorului
    execvp(shell, args.data());
}
```

```
// Codul de mai jos se executa doar in caz
// de eroare la execvp
perror("execvp");
_exit(1);
}
```

2. Implementarea mecanismului Time-Slicing (Preemptare) Sistemul emulează un scheduler de sistem de operare în user-space. Dacă o sarcină depășește cuanta de timp alocată (TIME_QUANTUM_MS), aceasta este suspendată folosind semnalul SIGSTOP.

Listing 6. executor.cpp: Logica de Preemptare (Liniile 127-145)

```
auto now = std::chrono::steady_clock::now();
auto elapsed = std::chrono::duration_cast<std::
    chrono::milliseconds>(now - start_time).
    count();

if (elapsed >= TIME_QUANTUM_MS) {
    // Verificam daca procesul inca ruleaza
    if (kill(-t.pid, 0) == 0) {
        std::cout << "[EXECUTOR] Time quantum
            expired. Suspending group...\n";

        // Trimitem SIGSTOP catre intregul grup
        // de procese (observati minusul)
        kill(-t.pid, SIGSTOP);
        t.suspended = true;

        // Returnarea sarcinii in coada pentru
        // executie ulterioara
        {
            LockGuard lock(queueMutex);
            userQueues[t.user].push_back(t);
        }
        queueCond.notify_all();
    }
}
```

Semnalul SIGCONT este utilizat ulterior pentru a relua execuția procesului "înghețat" exact din punctul în care a fost întrerupt, realizând un *context switch* la nivel de proces.

VIII. OPTIMIZĂRI ȘI GESTIONAREA RESURSELOR LA NIVEL DE SISTEM

Proiectul integrează tehnici avansate de programare de sistem pentru a asigura eficiența și stabilitatea, dincolo de simpla funcționalitate.

A. Sincronizare Non-Blocantă (Lock-Free)

Comunicarea dintre componenta Server (care recepționează cereri) și Planificator se realizează fără a bloca resursele partajate. Am utilizat o variabilă atomică (std::atomic<bool> reload_needed) pentru a semnaliza necesitatea

reîncărcării listei de sarcini. Această abordare elimină necesitatea unui *mutex* pentru operația de semnalizare, reducând latența sistemului și evitând inversiunile de prioritate specifice mecanismelor clasice de locking.

B. Managementul Ciclic al Thread-urilor (Detached Threads)

Deoarece serverul este proiectat să funcționeze pe perioade nedeterminate și să servească un număr variabil de clienți, gestionarea memoriei alocate firelor de execuție este critică. În locul abordării clasice *joinable*, am utilizat primitiva `pthread_detach` imediat după crearea fiecărui fir de execuție pentru clienți. Aceasta instruieste kernel-ul să elibereze automat resursele (stiva, descriptorii, TCB - Thread Control Block) imediat ce firul își încheie execuția, prevenind astfel scurgerile de resurse ("zombie threads") care ar putea duce la epuizarea memoriei serverului.

C. Robustete în fața Condițiilor de Cursă (Race Conditions)

O provocare majoră în gestionarea proceselor este situația în care un proces fii se termină extrem de rapid, înainte ca părintele să apuce să interacționeze cu el (să îi trimită semnale precum SIGSTOP). Implementarea din `executor.cpp` abordează această problemă prin utilizarea apelului `waitpid` cu flag-ul `WNOHANG` și tratarea explicită a erorii `ECHILD`. Aceasta permite sistemului să detecteze corect dacă un proces s-a terminat "natural" sau a fost deja curățat de sistemul de operare, evitând blocajele sau erorile de tip "No such process" care ar putea destabiliza executorul.

IX. LIMITĂRI ȘI DIRECȚII DE CERCETARE VIITOARE

Deși sistemul îndeplinește obiectivele funcționale, analiza performanței relevă anumite compromisuri arhitecturale care pot fi optimizate în iterațiile viitoare.

A. Complexitatea Algoritmului de Planificare

Implementarea actuală utilizează o parcurgere liniară a cozii la fiecare cuantă de timp, având o complexitate de $O(N)$. Pentru mii de sarcini, acest

lucru crește latența. O optimizare viitoare ar fi utilizarea unui *Min-Heap*, reducând timpul de selecție la $O(1)$ sau $O(\log N)$.

B. Scalabilitatea Modelului Thread-per-Client

Arhitectura curentă alocă un fir de execuție nativ (`pthread`) pentru fiecare client conectat. Deși simplă și robustă pentru un număr redus de utilizatori, această paradigmă întâmpină limitări de scalabilitate (cunoscută sub numele de "C10k problem") datorită overhead-ului de memorie al stivelor de thread și costului de context-switching. O evoluție naturală a sistemului ar implica tranziția către un model bazat pe evenimente (*Event-Driven Architecture*), utilizând mecanisme de I/O non-blocante precum `epoll` (Linux) sau `kqueue` (BSD), permițând unui număr redus de fire de execuție să deservească mii de conexiuni simultane.

C. Securitate și Izolarea Proceselor

Actuala izolare prin `setpgid` previne interferența semnalelor, dar nu restricționează accesul la sistemul de fișiere sau rețea. Integrarea cu mecanisme moderne de containerizare Linux Namespaces și cgroups ar oferi un nivel superior de securitate ("sandboxing"), similar platformelor precum Docker.

X. TESTARE ȘI VALIDARE EXPERIMENTALĂ

Pentru a valida robustețea și corectitudinea sistemului distribuit, am derulat o serie de scenarii de testare care vizează funcționalitatea de bază, comportamentul în condiții de concurență și precizia planificatorului.

A. Scenariul 1: Validarea Funcțională End-to-End

Am verificat fluxul complet al unei sarcini, de la inițiere până la finalizare.

- **Configurație:** Un client autentificat programează un script shell simplu (`echo "Test" > out.txt`) pentru execuție la $T_0 + 10s$.
- **Rezultat:** Serverul recepționează cererea, o stochează în XML. La momentul $T_0 + 10s$, Scheduler-ul extrage sarcina, iar Executorul lansează procesul.

- **Confirmare:** Fișierul de ieșire este creat corect, iar statusul sarcinii în istoricul clientului se actualizează din "WAITING" în "FINISHED (Code 0)".

B. Scenariul 2: Testarea Concurenței (Stress Test)

Pentru a evalua modelul *Thread-per-Client*, am simulat conectarea simultană a 5 instanțe de client.

- **Metodologie:** Fiecare client trimite simultan o cerere de Login urmată de o interogare Get All Tasks.
- **Observații:** Serverul a răspuns tuturor cererilor fără blocaje sau corupere de date iar thread-urile au fost eliberate corect (detached) la deconectare.

C. Scenariul 3: Verificarea Preemptării (Round-Robin)

Am testat capacitatea sistemului de a împărți echitabil procesorul între doi utilizatori (U1 și U2),

fiecare rulând câte un script intensiv (infinite loop).

- **Comportament Așteptat:** Executorul ar trebui să alterneze execuția celor două procese la fiecare cuantă de timp (TIME_QUANTUM).
- **Rezultat:** Log-urile serverului arată secvențe repetate de SIGSTOP trimis către procesul lui U1, urmat de SIGCONT pentru U2, și invers. Aceasta confirmă funcționarea corectă a mecanismului de *User-Space Time Slicing*.

XI. CONCLUZII

Sistemul demonstrează aplicarea conceptelor fundamentale de OS: IPC, multithreading și gestiunea proceselor. Mecanismul de preemptare asigură stabilitatea și distribuția echitabilă a resurselor într-un mediu multi-user.

[1] B. W. Kernighan, D. M. Ritchie, *The C Programming Language*, 2nd ed. (Prentice Hall, 1988).
 [2] R. Love, *Linux Kernel Development*, 3rd ed. (Addison-Wesley, 2010).
 [3] W. R. Stevens, S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. (Addison-Wesley, 2013).
 [4] R. Love, *Linux System Programming: Talking Directly to the Kernel and C Library*, 2nd ed. (O'Reilly Media, 2013).

[5] A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts Essentials*, 2nd ed. (Wiley, 2013).
 [6] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook* (No Starch Press, 2010).
 [7] A. S. Tanenbaum, H. Bos, *Modern Operating Systems*, 4th ed. (Pearson, 2014).
 [8] A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts*, 10th ed. (Wiley, 2018).